Lesson notes · for you

Measuring an agent

AI agents · University · Lesson 3

Time	Materials
20 min	Paper and a pen; A calculator or your phone's calculator; Your phone, for the lab

Learning goals

- Build a fixed test set with held-out wordings and names.
- Score exactly, and break the score down by group.
- Say honestly how sure a score is, and re-test after changes.

Activity: Write the agent's exam

1. Write (6 min): In pairs, write ten requests for a made-up timetable agent: five in plain wordings ("What does 4A have on Monday?") and five in unusual ones ("Monday, 4A: what's on?").
2. Answer key (4 min): On a separate sheet, write the exact call each request should produce, with the tool and every detail, before anyone plays the agent.
3. Swap and run (6 min): Swap requests with another pair. One partner plays the agent and may only use a list of phrases they agreed in advance; the other writes down each call the agent makes.
4. Score (4 min): Score with the answer key, exact match only, and give the plain and unusual groups separate scores. Discuss what failed and how sure ten requests can make you.

Use made-up classes and days only. A fixed list of phrases for the "agent" makes the unusual wordings fail, just as a small model does.

Discussion

- Why must the answers be written before the agent runs?
- What would you need to trust a score enough to deploy an agent?
- Which of your study tools has never been tested on your own language?

Check yourself

- One thing a fair test set must contain, and why 29 out of 32 is not exactly 91%.

Try it this week
Write ten test requests for an assistant you use, five in odd wordings, with the right answer for each written before you run them. Score them, and note which group failed more.

Worksheet

Measuring an agent

AI agents · University · Lesson 3

Name _____ Class _____ Date _____

1 **A test set uses only wordings the model was trained on, and it scores 100%. What does that show?**

◯ That it remembers what it was trained on ◯ That it is safe to give to users ◯ That it understands any wording

2 **Each test case is (group, want, got): its group, the call you wanted and the call the agent made, as dicts. Write score(cases) that returns a dict giving, for each group, the share of its cases where got is exactly want. (The agent's calls are given: no model runs here.)**

```
def score(cases):
    out = {}
    return out
```

Write your program:

3 **What does this program print?**

```
want = {"tool": "glossary",
    "args": {"term": "battery"}}
got = {"tool": "glossary",
    "args": {"term":
        "the term battery"}}
print(got["tool"] == want["tool"],
      got == want)
```

◯ False False ◯ True False ◯ True True

4 **Which belong in a good test set for an agent? Choose all that apply.**

☐ Brand-new requests each run, so the agent can't memorise them ☐ Wordings and names held out of training

☐ Requests in every language its users write ☐ The right answer for each, decided before the run

☐ Only the requests it got right last time

5 **Put the steps of an honest evaluation in order.**

☐ Score every case the same strict way

☐ Collect real requests, including new wordings and both languages

☐ Re-run the same set after every change

☐ Write the right answer for each, before running anything

☐ Run the agent on the whole set

☐ Break the score down and read every failure

✎ **Think of an AI tool you or your classmates rely on. What would a fair test set for it contain, and what score would you need before trusting it with something that matters?**

Check your answers

Measuring an agent

AI agents · University · Lesson 3

1 **That it remembers what it was trained on**

Trained wordings measure memory. Only held-out wordings and names show how it copes with requests it never saw: Darsi Tiny drops from 16/16 to 13/16.

2

```
def score(cases):
    hits, total = {}, {}
    for g, want, got in cases:
        total[g] = total.get(g, 0) + 1
        if got == want:
            hits[g] = hits.get(g, 0) + 1
    out = {}
    for g in total:
        out[g] = hits.get(g, 0) / total[g]
    return out
```

Scoring by group shows where an agent fails, which one overall number hides.

3 **True False**

The tools match (both glossary), so the first comparison is True; the details differ ("the term battery" against "battery"), so the whole calls are not equal.

4 **Requests in every language its users write; Wordings and names held out of training; The right answer for each, decided before the run**

Cover every language, include held-out wordings and names, and fix the answers in advance.

5 **Top to bottom: 4, 1, 6, 2, 3, 5**

Answers come before the run so you can't bend them to fit; strict scoring comes before reading failures; and the fixed set is re-run after every change to catch regressions.